Offline Medical AI Chatbot

Architecture, Technology Stack and Technical Guide

A document-based question-answering system built with Flask, Ollama and a local vector store. It uses Retrieval-Augmented Generation (RAG) to answer questions from your own medical PDF and TXT documents, and it runs completely offline with no API keys.

Item	Details
Application	Medical AI Assistant (Flask web application)
Approach	Retrieval-Augmented Generation (RAG)
AI models	Ollama nomic-embed-text (embeddings), Ollama llama3.2:3b (answers)
Vector store	Local NumPy file (knowledge_base.npz)
Runs on	Windows 10/11, Python 3.12, fully offline after setup
Document date	September 2026

Contents

1. Overview 3

What the system does . . . 3

Key design principles . . . 3

What is RAG? . . . 3

2. System Architecture 4

Layer 1: Presentation layer (web browser) . . . 4

Layer 2: Application layer (Flask server) . . . 4

Layer 3: AI and data layer . . . 5

3. Technology Stack 6

Why these choices . . . 6

4. Project Structure 7

File responsibilities . . . 7

5. Training Pipeline (Building the Knowledge Base) 9

How the progress bar works . . . 9

6. Question-Answering Pipeline 11

Step-by-step explanation . . . 11

7. Answer Cache 14

Vector store details . . . 14

8. User Interface 15

Home page . . . 15

Q&A; window . . . 15

Safeguards . . . 16

9. API Reference 17

Status codes . . . 17

10. Configuration Reference 18

11. Installation and Setup 19

Everyday start . . . 19

12. How to Use 19

Testing retrieval without the language model . . . 19

13. Troubleshooting 20

14. Limitations and Possible Improvements 20

1. Overview

The Medical AI Assistant lets a user upload medical documents (PDF or TXT), turn them into a searchable knowledge base, and ask questions in plain language. Answers are generated only from the uploaded documents, which keeps them grounded and reduces made-up information.

What the system does

- **Upload** one or many PDF/TXT documents through the web interface.
- **Train** a knowledge base: text is extracted, split into chunks, converted into embeddings and saved locally, with a live progress bar.
- **Ask** questions in a separate Q&A window and receive answers written by a local language model.
- **Reject** questions the documents do not cover instead of guessing.
- **Cache** answers so repeated questions return instantly.

Key design principles

Principle	How it is achieved
Fully offline	Ollama runs the AI models on the local computer and the vector store is a local file. No cloud service, account or API key is used.
Grounded answers	Only document chunks above a relevance threshold are given to the model, and the prompt forbids inventing facts.
Medical safety	The prompt forbids definitive diagnoses and medicine doses, and asks the model to flag emergencies and recommend professional care.
Reliability	Training runs in the background; a new knowledge base replaces the old one only after training succeeds.
Speed	Vectors are pre-normalised for fast search, and an answer cache returns repeated questions instantly.

What is RAG?

Retrieval-Augmented Generation combines two steps. **Retrieval** finds the passages in your documents that are most similar in meaning to the question. **Generation** gives those passages to a language model, which writes an answer based on them. This lets a small local model answer accurately about documents it was never trained on.

2. System Architecture

The system follows a three-layer architecture. Every layer runs on the same computer.

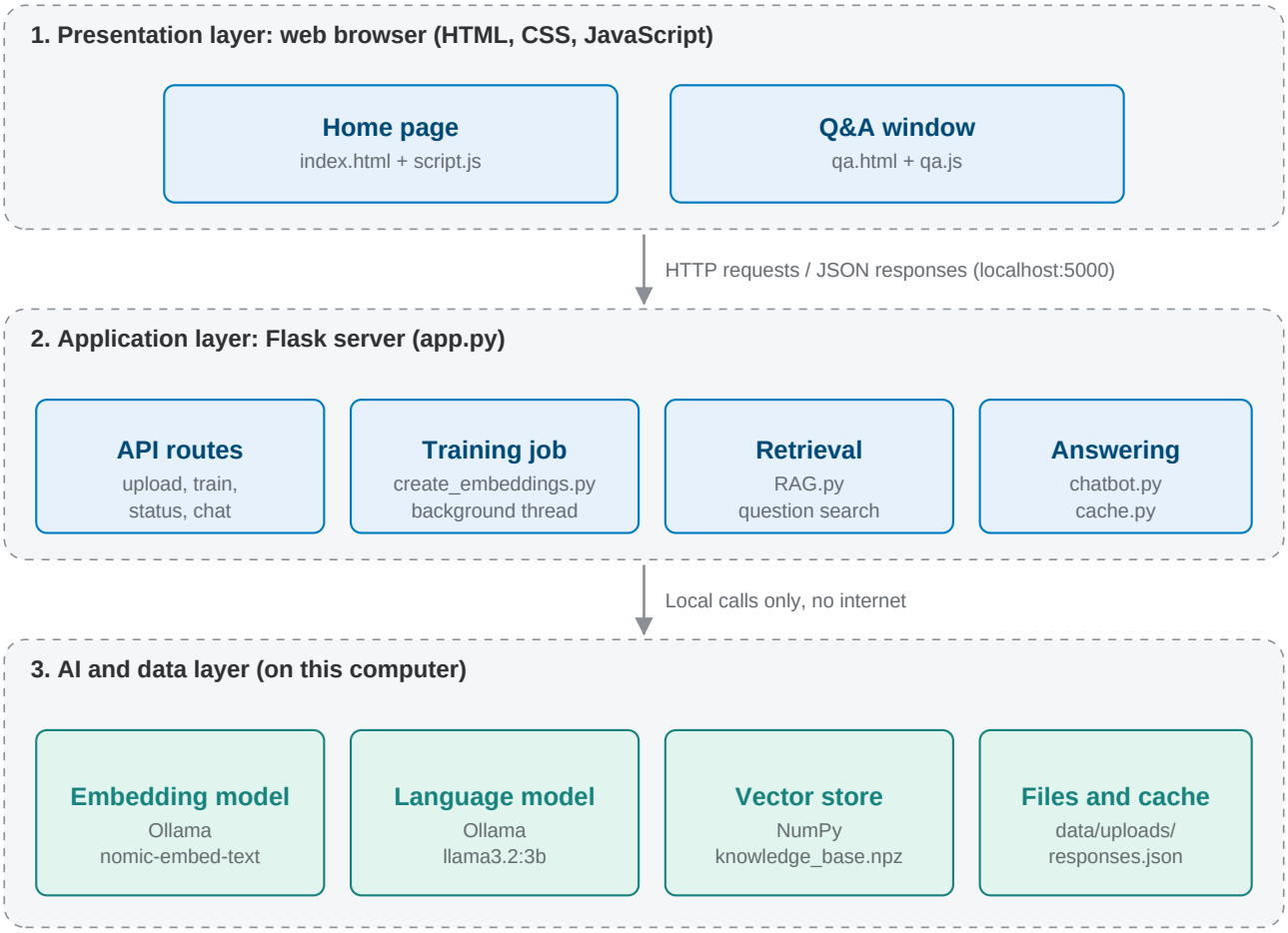


Figure 1. Three-layer architecture of the Medical AI Assistant

Layer 1: Presentation layer (web browser)

Two web pages built with plain HTML, CSS and JavaScript. They contain no business logic; they display information and send requests to the Flask server.

- **Home page** (index.html, script.js): upload and delete documents, start training, show the live progress bar and knowledge base summary, and open the Q&A window.
- **Q&A window** (qa.html, qa.js): a separate pop-up window where the user types questions, clicks example questions, and reads the answers.

Layer 2: Application layer (Flask server)

A Python Flask application (app.py) listening on http://127.0.0.1:5000. It serves the pages, exposes a JSON API and coordinates the AI components.

- **API routes** handle uploads, the document list, training, status and chat requests.
- **Training job** runs create_embeddings.py in a background thread and records its progress so the browser can poll it.
- **Retrieval** (RAG.py) finds the most relevant document chunks for a question.
- **Answering** (chatbot.py, cache.py) checks the cache, builds the prompt, calls the language model and stores the answer.

Layer 3: AI and data layer

- **Embedding model** (Ollama nomic-embed-text) converts text into 768-number vectors that capture meaning.
- **Language model** (Ollama llama3.2:3b) writes answers from the retrieved text.
- **Vector store** (knowledge_base.npz) holds every chunk's vector and text; search uses NumPy.
- **Files and cache**: uploaded documents in data/uploads/ and cached answers in data/cache/responses.json.

3. Technology Stack

All components are free and open source and run locally. Minimum versions come from backend/requirements.txt.

Layer	Technology	Version	Purpose in this project
Frontend	HTML5, CSS3, JavaScript (vanilla)	Browser built-in	Home page, Q&A window, progress bar, Bosch-inspired styling. No framework and no internet needed.
Web server	Python	3.12 recommended	Programming language for the whole backend.
Web server	Flask	>= 3.0	Serves pages and the JSON API; runs training in a background thread.
Configuration	python-dotenv	>= 1.0	Loads settings from backend/.env.
Document reading	pypdf	>= 5.0	Extracts text from each PDF page.
Document reading	fontTools	>= 4.50	Lets pypdf decode PDFs that embed special (CFF Type1) fonts.
Text processing	LangChain Core	>= 0.3	Document objects that carry text plus source and page metadata.
Text processing	LangChain Text Splitters	>= 0.3	RecursiveCharacterTextSplitter splits text into overlapping chunks.
AI connector	LangChain Ollama	>= 0.3	OllamaEmbeddings and ChatOllama talk to the local Ollama server.
AI runtime	Ollama	Latest	Runs AI models locally at http://localhost:11434.
Embedding model	nomic-embed-text	via Ollama	Produces 768-dimensional embeddings for chunks and questions.
Language model	llama3.2:3b	via Ollama	Small (3 billion parameter) model that writes the answers.
Vector store	NumPy	>= 1.26	Stores vectors in a compressed .npz file and computes cosine similarity.
Cache	Python standard library (json, hashlib, OrderedDict)	Built-in	Answer cache with least-recently-used removal and expiry.
Environment	Python venv	Built-in	Keeps project packages separate from the system.

Why these choices

- **Ollama** makes running models locally simple and keeps all data on the computer, which matters for medical information.
- **nomic-embed-text** is a compact, high-quality embedding model that runs well on ordinary hardware.
- **llama3.2:3b** is small enough to run on a laptop while still writing clear answers from supplied text.
- **A NumPy vector store** needs no database server. For typical document collections (thousands to tens of thousands of chunks), a single matrix multiplication searches everything in milliseconds.
- **Flask and vanilla JavaScript** keep the project small and easy to understand, with no build tools.

4. Project Structure

```
Medical_Chatbot_Ollama_Flask/
|-- README.md
|-- .gitignore
|-- backend/
|   |-- app.py                  Flask server, API routes, training job
|   |-- config.py               All settings (reads .env)
|   |-- requirements.txt        Python packages
|   |-- .env.example            Example settings
|   |-- data/
|   |   |-- uploads/            Uploaded PDF/TXT files
|   |   |-- vector_store/       knowledge_base.npz (created by training)
|   |   `-- cache/              responses.json (created by first answer)
|   `-- src/
|       |-- __init__.py
|       |-- create_embeddings.py  Training pipeline
|       |-- vector_store.py       Save, load and search vectors
|       |-- RAG.py                Retrieval step
|       |-- chatbot.py            Prompt + answer generation (+ cache)
|       `-- cache.py              Question -> answer cache
`-- frontend/
    |-- templates/
    |   |-- index.html          Home page
    |   `-- qa.html             Q&A window
    `-- static/
        |-- css/style.css       Styling for both pages
        `-- js/
            |-- script.js       Home page logic
            `-- qa.js           Q&A window logic
```

File responsibilities

File	Responsibility
app.py	Creates the Flask app, defines all routes, runs training in a background thread with progress tracking, clears the cache after training, and adds a version number to CSS/JS links so browsers always load the latest files.
config.py	Central settings: Ollama URL and models, embedding dimension, chunk size and overlap, TOP_K, relevance threshold, cache settings, folder paths and upload size limit.
create_embeddings.py	Reads documents, skips pages without text, splits text into chunks, creates embeddings in batches of 50, reports progress, and saves the knowledge base.
vector_store.py	Normalises and saves vectors with their text in one .npz file (written safely to a temporary file first), reloads it only when it changes, and searches by cosine similarity.
RAG.py	Retriever class: checks the knowledge base, embeds the question, searches, drops chunks below the threshold and removes duplicates. Can be run alone to test retrieval.
chatbot.py	ask() checks the cache, then retrieval, prompt building and generation with llama3.2:3b; stores the answer in the cache.
cache.py	Stores answers keyed by the normalised question and a fingerprint of the knowledge base and settings; limits size and handles expiry.
index.html / script.js	Home page layout and behaviour: upload, delete, train, progress polling, knowledge base summary, opening the Q&A window.
qa.html / qa.js	Q&A window layout and behaviour: example questions, Enter to send, typing indicator, answer display.

File	Responsibility
style.css	Shared design: colour band, cards, progress bar, chat bubbles, responsive layout for small screens.

5. Training Pipeline (Building the Knowledge Base)

Training turns uploaded documents into a searchable knowledge base. It starts when the user clicks **Train Knowledge Base** on the home page.

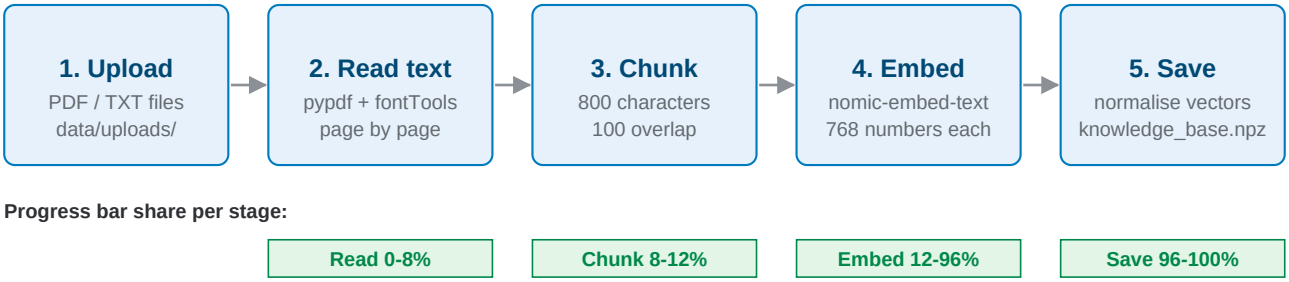


Figure 2. The five training steps and their share of the progress bar

Step 1: Upload documents

The user selects one or more PDF or TXT files. The browser sends them to **POST /upload**. The server cleans each file name (keeping non-English characters), rejects other file types, renames duplicates instead of overwriting them (for example report (1).pdf), and saves the files in backend/data/uploads/. The maximum upload size is 50 MB by default.

Step 2: Read the text

When training starts, **POST /train** launches a background thread and immediately returns, so the page stays responsive. The thread first checks that Ollama is reachable and that the embedding model returns 768 numbers. It then reads every uploaded file: PDFs page by page with pypdf (fontTools helps with embedded fonts), and TXT files as UTF-8 text. Pages with fewer than 20 characters of text, usually scanned images, are skipped and reported. Each page keeps its file name and page number as metadata.

Step 3: Split into chunks

Long pages are too big to search precisely, so the text is split into chunks of about **800 characters** with **100 characters of overlap** between neighbouring chunks. The overlap prevents a sentence from being cut off at a chunk boundary and losing its meaning. The splitter prefers to break at paragraph breaks, then line breaks, then sentence ends, then spaces.

Step 4: Create embeddings

Each chunk is sent to the **nomic-embed-text** model through Ollama, in batches of 50. The model returns an **embedding**: a list of 768 numbers that represents the meaning of the text. Chunks about similar topics get similar numbers, even when they use different words. This is the slowest step, so it gets most of the progress bar (12% to 96%), and the page shows how many chunks are done and roughly how long is left.

Step 5: Save to the embedding database

All vectors are scaled to length 1 (normalised) so that later similarity search is a single fast multiplication. The vectors, the chunk texts, their file names and page numbers, and information about the run (embedding model, number of chunks, date and time) are written to **backend/data/vector_store/knowledge_base.npz**. The file is first written under a temporary name and then swapped in, so if training fails, the previous knowledge base is kept unchanged. Finally, the answer cache is cleared because old answers may no longer match the new documents.

How the progress bar works

While training runs, the home page asks **GET /train/status** every 0.7 seconds. The server replies with the current stage, the percentage, the number of chunks done and the time elapsed. If the page is refreshed during training, it picks up the progress again automatically.

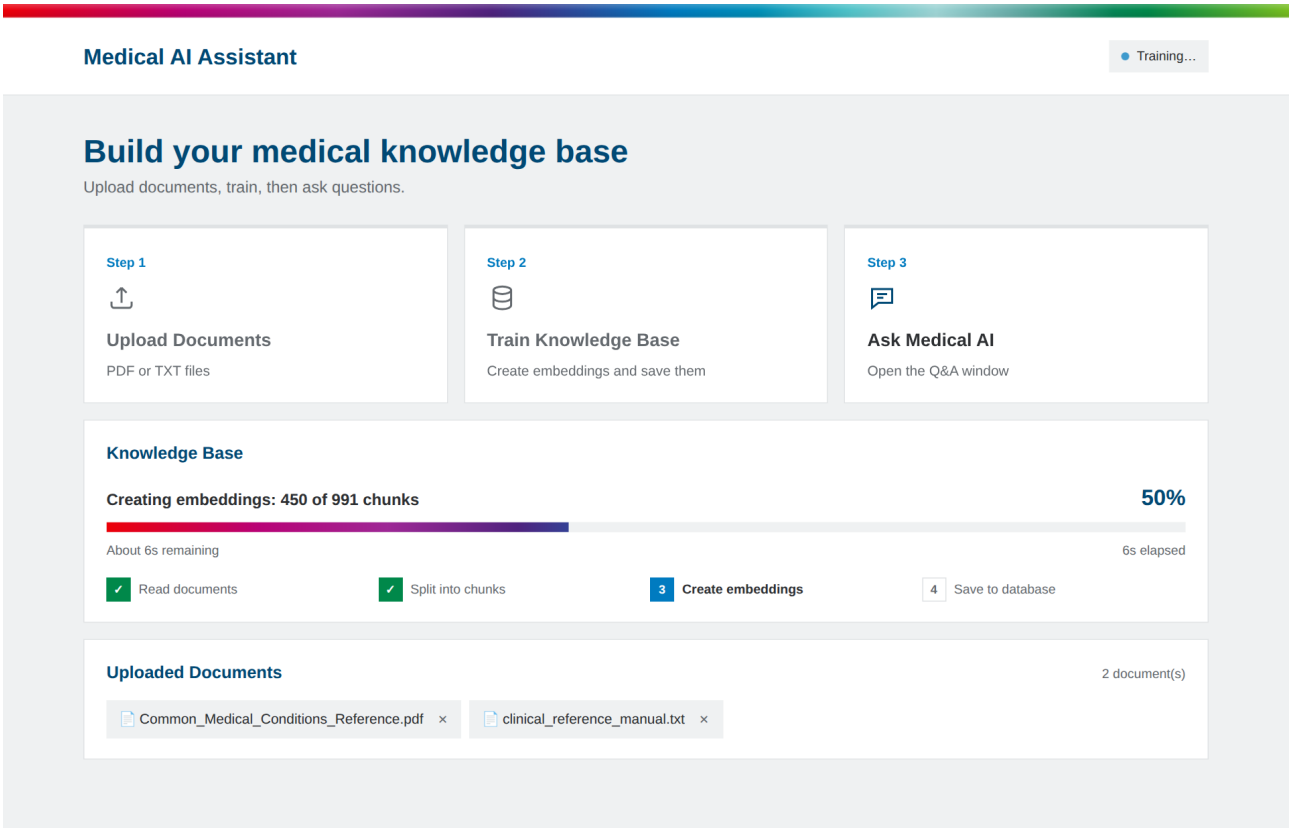


Figure 3. Training in progress: stage list, percentage, chunks done and time estimate

6. Question-Answering Pipeline

This is what happens from the moment the user presses Enter in the Q&A window until the answer appears.

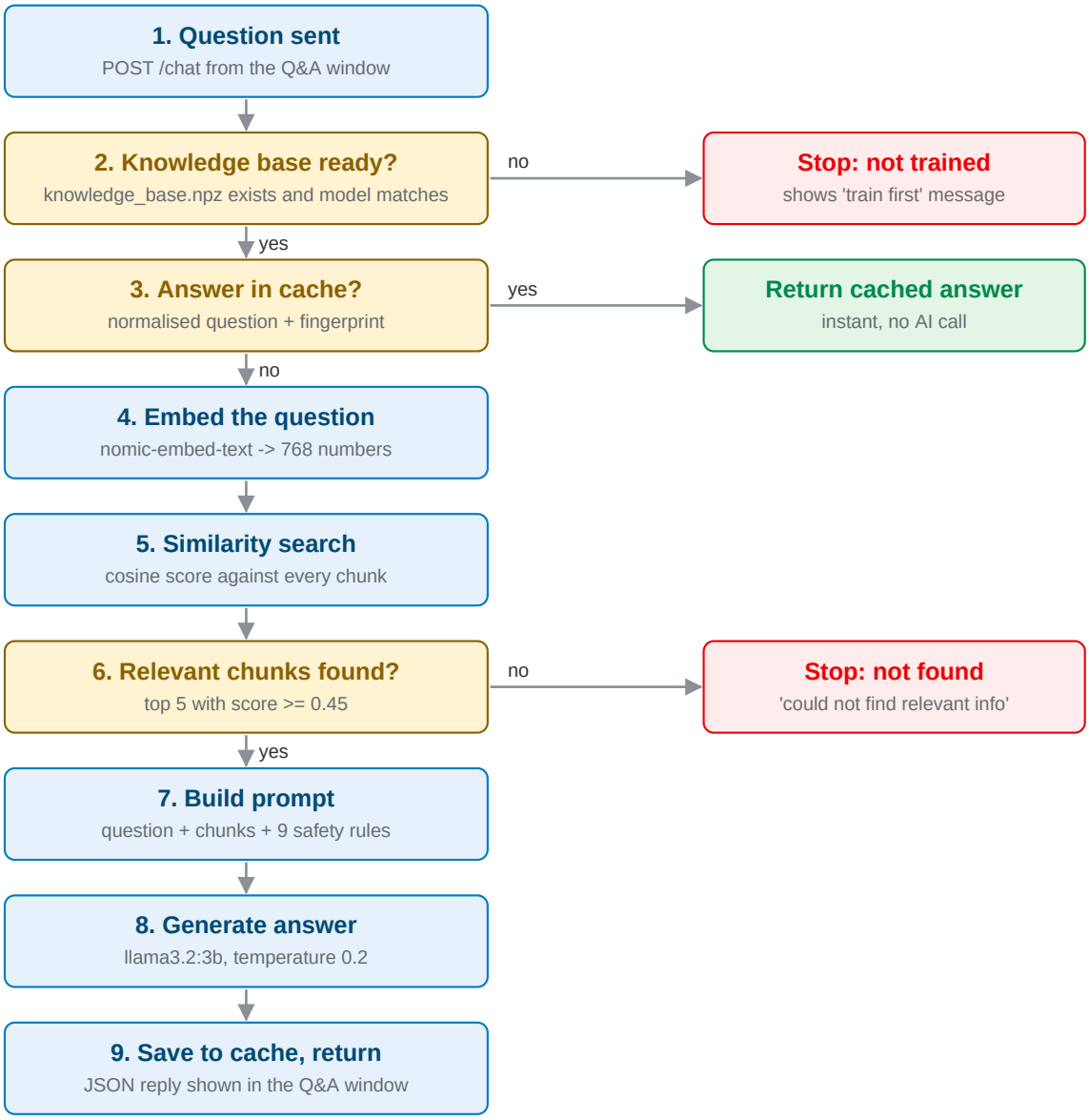


Figure 4. Question-answering flow with its three early exits

Step-by-step explanation

Step 1: Question sent

qa.js sends the question to **POST /chat** as JSON and shows a typing indicator. Empty questions are rejected.

Step 2: Check the knowledge base

The server checks that knowledge_base.npz exists and was built with the current embedding model. If not, it replies with 'The knowledge base has not been trained yet', and the AI is never called.

Step 3: Check the answer cache

The question is normalised (lower case, extra spaces and a trailing question mark removed) and combined with a fingerprint of the knowledge base and settings. If the same question was answered before with the same knowledge base, the saved answer is returned instantly. See section 7.

Step 4: Embed the question

The question is turned into a 768-number vector with the same nomic-embed-text model used for the documents, so questions and chunks can be compared directly.

Step 5: Similarity search

The question vector is compared with every chunk vector using **cosine similarity**, a score from about 0 (unrelated) to 1 (same meaning). Because all vectors were normalised during training, this is one NumPy matrix multiplication. The 5 highest-scoring chunks (TOP_K) are selected.

Step 6: Relevance filter

Chunks scoring below the **relevance threshold** (0.45 by default) are removed, along with exact duplicates. If nothing is left, the documents do not cover the question and the server replies 'I could not find relevant information about this in the uploaded documents' without calling the language model. This is what stops the chatbot from answering off-topic questions such as 'How do I repair a car?'.

Step 7: Build the prompt

The question and the remaining chunks are inserted into a prompt template that tells the model to use only this context and to follow nine rules:

1. Answer clearly and simply.
2. Do not present a definitive diagnosis.
3. Describe possible causes or conditions only as possibilities.
4. Give precautions only when the context supports them.
5. Do not prescribe medicines or dosages.
6. Advise urgent professional care if symptoms could indicate an emergency.
7. Say when the documents do not contain enough information instead of guessing.
8. Encourage consultation with a qualified healthcare professional.
9. Give only the answer, without listing sources, file names, page numbers or references.

Step 8: Generate the answer

The prompt is sent to **llama3.2:3b** through Ollama with a low temperature (0.2), which keeps answers focused and consistent rather than creative.

Step 9: Save and return

The answer is stored in the cache and sent back to the browser, where qa.js shows it in the chat. Errors are never cached, so a failed request is retried properly next time.

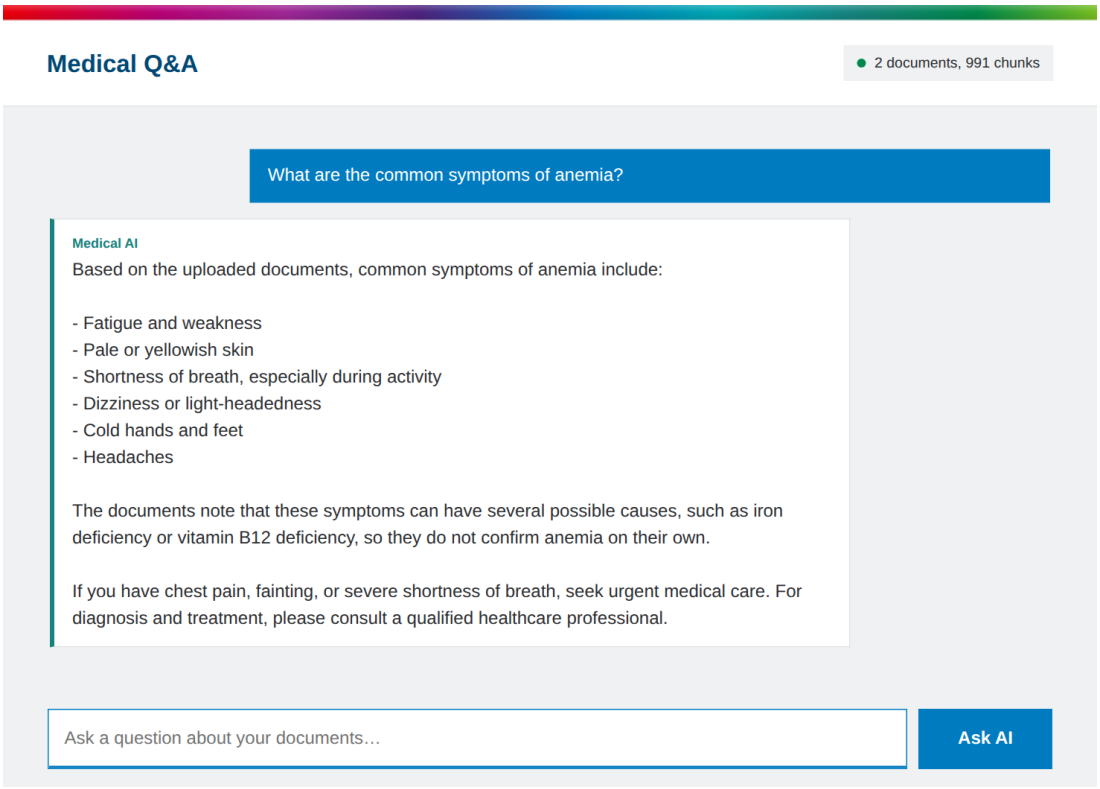


Figure 5. The Q&A window showing only the answer (sample answer text)

7. Answer Cache

Generating an answer with the language model takes several seconds. Many users ask the same questions, so the system saves answers and reuses them.

Aspect	Behaviour
Where	backend/data/cache/responses.json (kept when the app restarts)
Question matching	Questions are normalised first: 'What is anemia?' and 'what is anemia' share one entry.
Fingerprint	Each entry is tied to the training date and time, the number of chunks, both model names, TOP_K, the relevance threshold and the prompt version. If any of these change, old entries simply stop matching.
Automatic clearing	The whole cache is cleared after every successful training run.
Size limit	500 answers by default; the least recently used answer is removed first.
Expiry	168 hours (7 days) by default; 0 means answers never expire.
Monitoring	GET /cache shows entries, hits and misses; POST /cache/clear empties it.

Vector store details

The knowledge base is one compressed NumPy file containing three parts: **vectors** (a matrix with one row of 768 numbers per chunk), **metadata** (the text, file name and page number of each chunk) and **info** (embedding model, chunk count, dimension and creation time). The file is loaded into memory once and reloaded only when it changes on disk. On load, the system checks that the number of vectors matches the number of chunks; a damaged file produces a clear 'rebuild the knowledge base' message.

8. User Interface

The interface uses a Bosch-inspired colour palette: a multi-colour band at the top of each page (red, magenta, purple, blue, turquoise and green), Bosch blue for actions, dark blue for headings, green for success and red for errors. The Bosch Sans font is used if it is installed; otherwise Segoe UI or Arial.

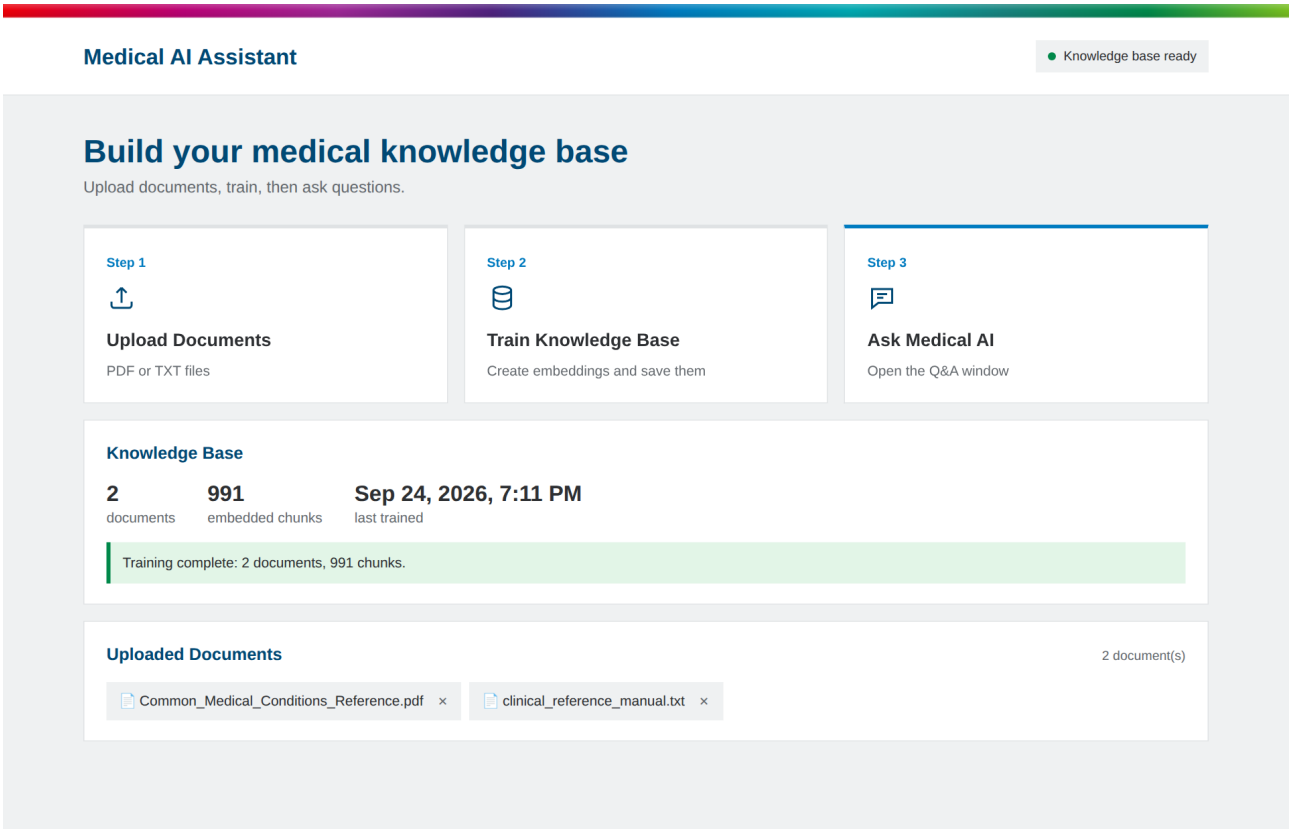


Figure 6. Home page after training

Home page

- **Step 1: Upload Documents** opens a file picker for one or more PDF/TXT files.
- **Step 2: Train Knowledge Base** asks for confirmation, then starts training and shows the progress bar in the Knowledge Base panel. The colour band at the top shimmers while training runs.
- **Step 3: Ask Medical AI** opens the Q&A window as a pop-up (or in the same tab if pop-ups are blocked).
- **Knowledge Base panel** shows the number of documents, embedded chunks and the last training time.
- **Uploaded Documents** lists files; the x button deletes a file (train again afterwards to remove it from answers).
- The next recommended step is highlighted with a blue top border.

Q&A window

- The header shows the knowledge base size (for example '2 documents, 991 chunks').
- Three example questions can be clicked to ask them instantly.
- Press Enter to send and Shift+Enter for a new line.
- A typing indicator appears while the answer is generated; only the answer text is shown.

Safeguards

- CSS and JavaScript links carry a version number, so browsers always download the newest files.
- If an outdated index.html or qa.html is mixed with newer scripts, a red banner explains which file to replace.
- All text from the server is inserted as plain text, so document content cannot inject HTML into the page.

9. API Reference

All endpoints are served by app.py at http://127.0.0.1:5000 and return JSON unless noted.

Method	Path	Purpose	Main response fields
GET	/	Home page (HTML)	-
GET	/qa	Q&A window (HTML)	-
POST	/upload	Upload files (form field 'files')	success, message, uploaded, renamed, rejected
GET	/documents	List uploaded files	documents
DELETE	/documents/<name>	Delete one uploaded file	success, message, documents
POST	/train	Start background training (returns 202)	success, message
GET	/train/status	Live training progress	state, stage, current, total, percent, message, elapsed_seconds, result, error
GET	/status	Knowledge base summary	trained, documents, chunks, created_at, embedding_model
POST	/chat	Ask a question: {"message": "..."}	success, response, sources, cached
GET	/cache	Cache statistics	enabled, entries, max_entries, ttl_hours, hits, misses
POST	/cache/clear	Empty the answer cache	success, message

Status codes

Code	Meaning
200	Success
202	Training started
400	Invalid request, for example an empty question or unsupported file type
404	Document not found
409	Training already running, or knowledge base not trained yet (chat)
413	Upload larger than MAX_CONTENT_LENGTH_MB
500	Unexpected error; the message explains the cause (for example Ollama not running)

10. Configuration Reference

Settings live in backend/.env (copy .env.example to create it). The defaults work without a .env file. Restart python app.py after changing a setting.

Setting	Default	Description
OLLAMA_BASE_URL	http://localhost:11434	Address of the local Ollama server
OLLAMA_LLM_MODEL	llama3.2:3b	Model that writes answers
OLLAMA_EMBEDDING_MODEL	nomic-embed-text	Model that creates embeddings (re-train after changing)
EMBEDDING_DIMENSION	768	Expected embedding size; must match the embedding model
CHUNK_SIZE	800	Characters per chunk
CHUNK_OVERLAP	100	Characters shared between neighbouring chunks
TOP_K	5	Maximum number of chunks given to the model
RELEVANCE_THRESHOLD	0.45	Minimum similarity score (0 to 1). Lower it if relevant questions get 'not found'; raise it if off-topic questions get answers.
CACHE_ENABLED	true	Turn the answer cache on or off
CACHE_MAX_ENTRIES	500	Maximum cached answers
CACHE_TTL_HOURS	168	Hours before a cached answer expires (0 = never)
FLASK_DEBUG	true in .env.example (false if unset)	Flask debug mode; keep it for local development only
MAX_CONTENT_LENGTH_MB	50	Maximum upload size per request

11. Installation and Setup

Internet is needed only for steps 1 and 3 (downloading Ollama models and Python packages). After that, everything runs offline.

Step 1: Install Ollama and pull the models

```
ollama --version
ollama pull nomic-embed-text
ollama pull llama3.2:3b
ollama list
```

Step 2: Create and activate a virtual environment (PowerShell, in the project folder)

```
py -3.12 -m venv venv
.\venv\Scripts\Activate.ps1

# If scripts are blocked:
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\Activate.ps1
```

Step 3: Install the Python packages

```
cd backend
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Step 4: (Optional) create backend/.env from .env.example and adjust settings.

Step 5: Start the application

```
python app.py
# then open http://127.0.0.1:5000 in the browser
```

Everyday start

```
.\venv\Scripts\Activate.ps1
cd backend
python app.py
```

12. How to Use

1. Make sure Ollama is running.
2. Open http://127.0.0.1:5000.
3. Click **Upload Documents** and choose PDF or TXT files.
4. Click **Train Knowledge Base** and wait for the progress bar to reach 100%.
5. Click **Ask Medical AI** to open the Q&A window.
6. Type a question and press Enter, or click an example question.
7. After adding or deleting documents, click Train Knowledge Base again.

Testing retrieval without the language model

From the backend folder, this command shows which chunks a question retrieves, their scores and whether each passes the threshold. It is the easiest way to tune RELEVANCE_THRESHOLD.

```
python src/RAG.py "What are the symptoms of anemia?"
```

13. Troubleshooting

Problem	Cause	Solution
'Could not reach Ollama'	Ollama is not running or a model is missing	Start Ollama; run ollama list and pull missing models.
'The knowledge base has not been trained yet'	No knowledge_base.npz yet	Upload documents and click Train Knowledge Base.
Relevant questions return 'could not find'	Threshold too strict for your documents	Run python src/RAG.py with the question and lower RELEVANCE_THRESHOLD slightly (for example 0.40).
Off-topic questions get answers	Threshold too loose	Raise RELEVANCE_THRESHOLD (for example 0.55).
Pages skipped during training	Scanned PDF without a text layer	Run OCR on the PDF before uploading.
fontTools warnings in the terminal	PDF uses embedded CFF fonts	pip install fonttools (already in requirements.txt), then train again.
Page looks wrong or mixes old and new parts	Old HTML/CSS/JS files or browser cache	Replace the whole frontend folder, restart the app and press Ctrl+F5.
Q&A window does not open	Browser blocks pop-ups	Allow pop-ups for 127.0.0.1, or open http://127.0.0.1:5000/qa.
'Embedding dimension mismatch'	Embedding model changed	Set EMBEDDING_DIMENSION to the model's size and train again.
Old answers after changing documents	Training not re-run	Click Train Knowledge Base; this also clears the cache.
PowerShell will not activate venv	Script execution policy	Run Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass first.

14. Limitations and Possible Improvements

Current limitation	Possible improvement
Scanned PDFs cannot be read	Add OCR (for example Tesseract) during training.
Training re-embeds all documents every time	Embed only new or changed files (incremental training).
Exact-match cache only	Add a semantic cache that reuses answers for questions with very similar meaning.
Single user, local computer	Run behind a production server (for example Waitress) with user authentication for shared use.
No conversation memory	Include recent chat history in the prompt for follow-up questions.
Small 3B model	Use a larger Ollama model on stronger hardware for richer answers.

This guide describes the software architecture. The chatbot provides information from uploaded documents and is not a substitute for professional medical advice.